Project Milestone P2

Analysis and High-Level Design

Password Manager

Team	Team 09, section .003
Document version	0.2
Date	10/30/2026
Repository URL	https://github.com/MVT04/3354-Team-09-Project
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	[one member only]

Team member	NetID	GitHub username
Samantha Escalada	sxe220036	esamalada
Shubhi Kohli	dal703399	skkohli1330
Dongho Lee	dxl240021	dongholee96s
Jacob Weikum	dal796652	JacobWeikum
Derek Chai	dwc230000	Derek-Chai
Michael Taylor	mvt210000	MVT04

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change
0.1	10/18/2026	Team 09	P1 baseline (copy your last P1 row)
0.2	10/30/2026	Team 09	P2 baseline: analysis and high-level design added
0.3	11/07/2026	Team 09	Scope revised following feasibility analysis and instructor feedback; 2FA, Web App, and autofill implementation removed for the planned implementation scope. Related requirements, use cases, and architecture updated.
1.0	11/08/2026	Team 09	Design rationale, prototype evidence, traceability, project plan, and open questions were filled out and built upon.

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Derek Chai	Section 5.2 scenario tables and non-trivial step analysis for UC-01 and UC-02	Commit 2140524 (Derek_Chai_5.2.pdf), PR #8 (5.2 markdown and diagram sources)	PR #14
Michael Taylor	Transfer sections 1.1, 2.6, 3.2 and context diagram to repository. Updated context diagram. Section 7.1.	Commits: b5232fb, 08de975, PR #2, PR #7	PR #5, 6
Dongho Lee	5.1, 6.3,9.1, 11 - Domain Model and domain model diagram and matrix, Component View and component diagram and matrix, Extended Traceability Matrix.	Commits: a01bf88 (PR #1), ed7b321 (PR #3)	PR #2, 7, 8, 10,11,13,15
Shubhi Kohli	4.2, 4.3, 6.4,6.5,6.6 - UML Deployment view diagram, extended use case for UC-03 and self review log.	6.4, 6.5, 6.6 - commit hash: 02c7ca3 4.2 Use Case (before change) - 640f613 4.3 review log - c1fec9e	
Jacob Weikum	5.3, 0.2, 8, 10.2, 10.3, Styling for whole document, and updated old use case diagram	5.3 - commit 0aec4f6 0.2 - commit da54d9e 8 - commit ef67a13	

		10.2 - commit 7dec8ef 10.3 - commit e370137	
Samantha Escalada	4.1, 4.2, 5.2.3, 6.2, 7.2, 9.2	Commits d7d06ac b76249f	PR #12, PR #3

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Remove 2FA intention	It would require learning a new library with complex topics. Additionally it would require the use of google authenticator, a mobile app. We don't have the time or experience to make this a feasible feature.	Removal of 2FA features. Lowers our scope and makes our project much more feasible.	10/30/2026
CH-02	Remove web application intention	A web application cannot interact with the file system due to safety reasons. Thus in order to get around this for external back up storage, we would need to create a server relationship, which we don't have the experience to do.	Makes our program one application on the computer. Unable to do autofilling anymore. Lowers our scope and makes our project much more feasible.	10/30/2026
CH-03	Remove automatic autofill intention	It is not possible to interact with the web browser without being a web application. Furthermore, it would be very hard to implement even as a web application since websites have their own login page formats which would make it hard to identify inputs.	Not necessary to be a web application anymore. Lowers our scope and makes our project much more feasible.	10/30/2026

1 Introduction

1.1 Purpose of this document and of the system

This document is intended to be an overview of the proposal and requirements for the development of a password / identity and security management system. The document is intended for the project's team as guidance, and for the relevant stakeholders (Dr.

Seker). It provides a common understanding of the proposed system's objectives, design features, deliverables, and a planned outline. It will function as a reference point throughout the software engineering process.

The system aims to solve the issues users encounter when creating, organizing, and managing passwords, and other online account information across many different services. Remembering unique passwords with varying requirements may be difficult, leading to users choosing weak ones, or storing sensitive information in insecure areas. This proposed system will serve as a centralized application for users to enhance their password security, while keeping the process simple and convenient.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
A password storage system	Identity breach warning – Not feasible because of timing limitations. This is also not feasible because of technical limitations.
A password generator that generates passwords in accordance with requirements set by the user.	Multi-device use - Not feasible because of timing and technological limitations
Credential encryption	Sharing login information with others trusted by user - Not feasible because requires a lot of security
Password strength evaluator	Blocking phishing attempts- although security and login related, not the main intent of the program. It is also difficult to do in the time we have.

They were excluded mostly because they were not feasible given the amount of time we have to complete this project. Additionally, some of these items require more technical knowledge to execute securely.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
Credentials	Whatever fields of entry a particular website needs to authorize the user's access to its content. This includes, but is not limited to, usernames as well as passwords.
Strong Password	A password that is ~16 characters long, unique (ie a different password for each service), and random
Vault	The encrypted storage method where generated credentials and saved
Password Manager	Not the same thing as a vault. It *includes* the vault, but it is the overarching app that supports using and storing the passwords.

1.4 References

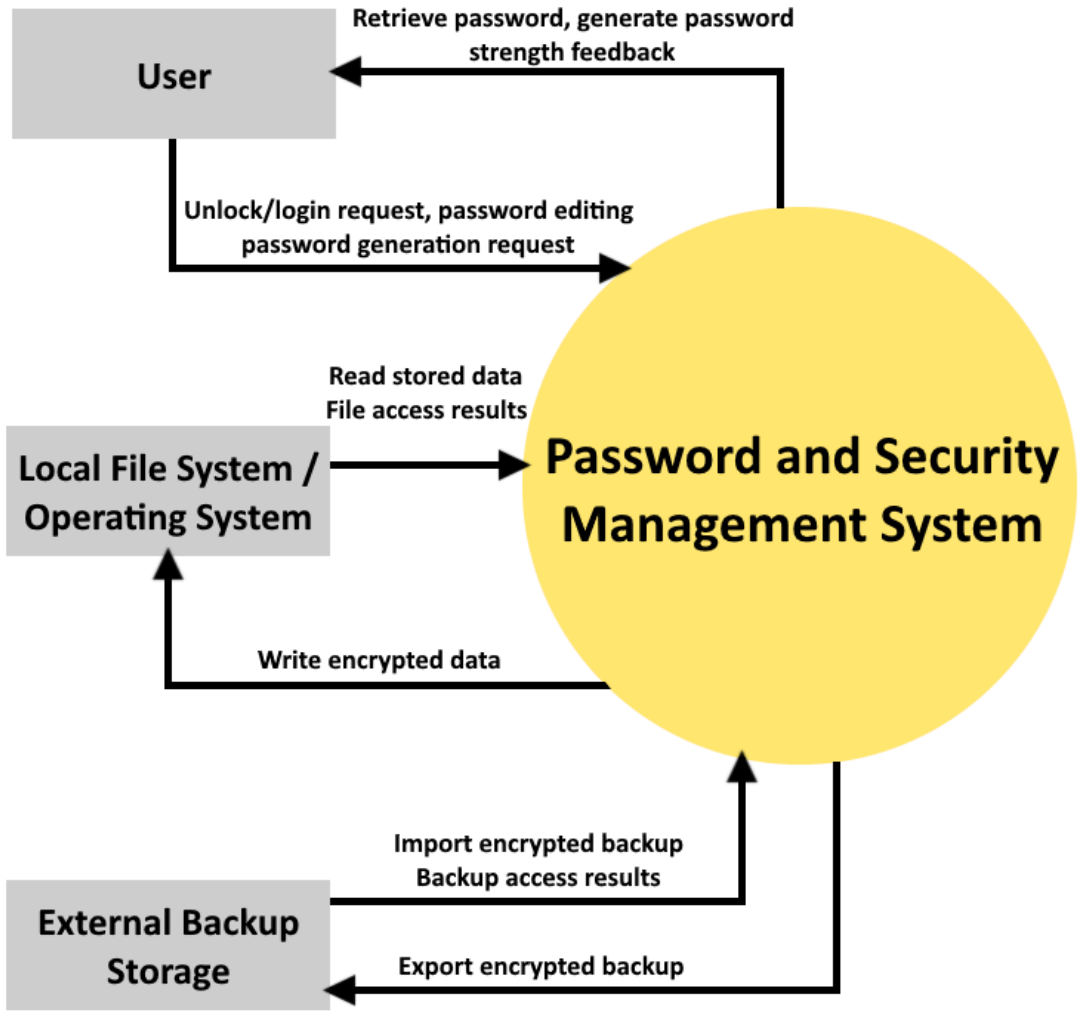
[R1]

[R1] An existing password management service

[R2] Guidelines for strong passwords as set by the Cybersecurity and Infrastructure Security Agency

2 Overall Description

2.1 Product perspective and context



The system boundary points out the functionality developed by the project team for the password and security management system, which includes password management, credential encryption, and localized vault handling. The diagram includes optional external integrations: external backup storage.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Everyday users	The people who store or want to generate passwords for different sites.	For it to work fast and not buffer or create the same password twice	Interviewed friends and siblings oct 13
People who are nontechnical	The people who also want to store / generate passwords	For it to be easy and straight forward to use	Interviewed parents Oct 16
Developers	The people creating the product	Needs it to be testable and maintainable.	Team conversation

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Professor	The one judging our outcome of our project	A workable deliverable project that meets the requirements made with team compatibility	DIscussed in class

2.3 User characteristics

Everyday Users - students, adults, professionals, anyone who manages multiple accounts for websites, apps, people who rely on the autofill passwords. They know how to login and add a browser extension. We wouldn't expect them to know the importance of a master password. They probably use a phone or laptop for the system, using it when signing onto a website during whatever they are doing. So the system should work fast.

Nontechnical users - parents, older users, anyone who's less comfortable with basic apps but wants to keep their information private and safe and who uses a phone. We can't assume them to know two factor authentication, or know what a recovery key is. They probably use a small phone device and may need larger text, simple wording and need it to have a simple setup. If it takes too long they may get frustrated.

Professor - Professor would evaluate if our systems follow correct practice. You would know modeling and secure storage patterns but you can't assume to know our design decisions so our documentation will state them explicitly. You would interact with our system through our documentation, source code and demos than daily use.

2.4 Constraints

C1 the system must be delivered by the end of our fall 2026 semester - source our course schedule

C2 Systems must only use free open source services (no paid hosting) - source team decision

C3 System must be demonstrable and professor can access it - source instructor requirements

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	Users will remember their master password or recovery key	The vault is unrecoverable since we can't decrypt without authorization
A2	Users are willing to adopt a dedicated password manager toll instead of reusing passwords or relying on autofill	Adoption is low, so the usability requirements don't deliver value and need a rethink
A3	Team members all have the time and skills needed to deliver the scope	Some additions may need to get cut that are more low priority or the scope gets reduced
D1	Free hosting stays available through the semester	Our website and the whole system won't be available to use.

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible	The system will be developed using programming languages and frameworks that all team members are familiar with.
Operational	Feasible	The problem addressed in section 1.1, the intended users, and proposed functionality.
Schedule	Feasible	The team will assign roles and track progress. See milestone plan and submission deadline.
Economic / resource	Feasible	The project is planned around the teams' existing computers and development tools.

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one "shall" sentence	Priority	Source	Acceptance criterion
FR-01	The system shall allow a user to unlock an existing vault using valid login information	MUST	Everyday users	A tester can enter valid login information and gain access to the vault
FR-02	The system shall allow an authorized user to store a new credential in the vault	MUST	Everyday users/R1	A tester can enter a credential. Save it , and retrieve the same credential later.
FR-03	The system shall allow an authorized user to retrieve a stored credential from the vault	MUST	Everyday users	A tester can select a stored credential and view the saved account information.
FR-04	The system shall allow an authorized user to edit an existing credential	MUST	Everyday users	A tester can modify a stored credential and confirm that the updated values are retained
FR-05	The system shall generate a password according to supported requirements selected by the user	MUST	Everyday users/R2	A tester can select supported password requirements and receive a password that satisfies them.
FR-06	The system shall display a strength evaluation for a password entered by the user	SHOULD	Everyday users/R2	A tester can enter a password and receive a strength evaluation.
FR-07	The system shall allow an authorized user to delete a stored credential	MUST	Everyday users	A tester can delete stored credential and confirm that it is no longer available
FR-08	The system shall provide a matching stored credential to a supported web browser for autofill	SHOULD	Everyday users/R1	A tester can request autofill and confirm that the browser receives the matching credential

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-09	The system shall save an encrypted backup of the vault	COULD	Everyday users/ R1	A tester can create an encrypted backup file
FR-10	The system shall restore the vault from an encrypted backup	COULD	Everyday users / R1	A tester can select a backup file and restore the saved credentials

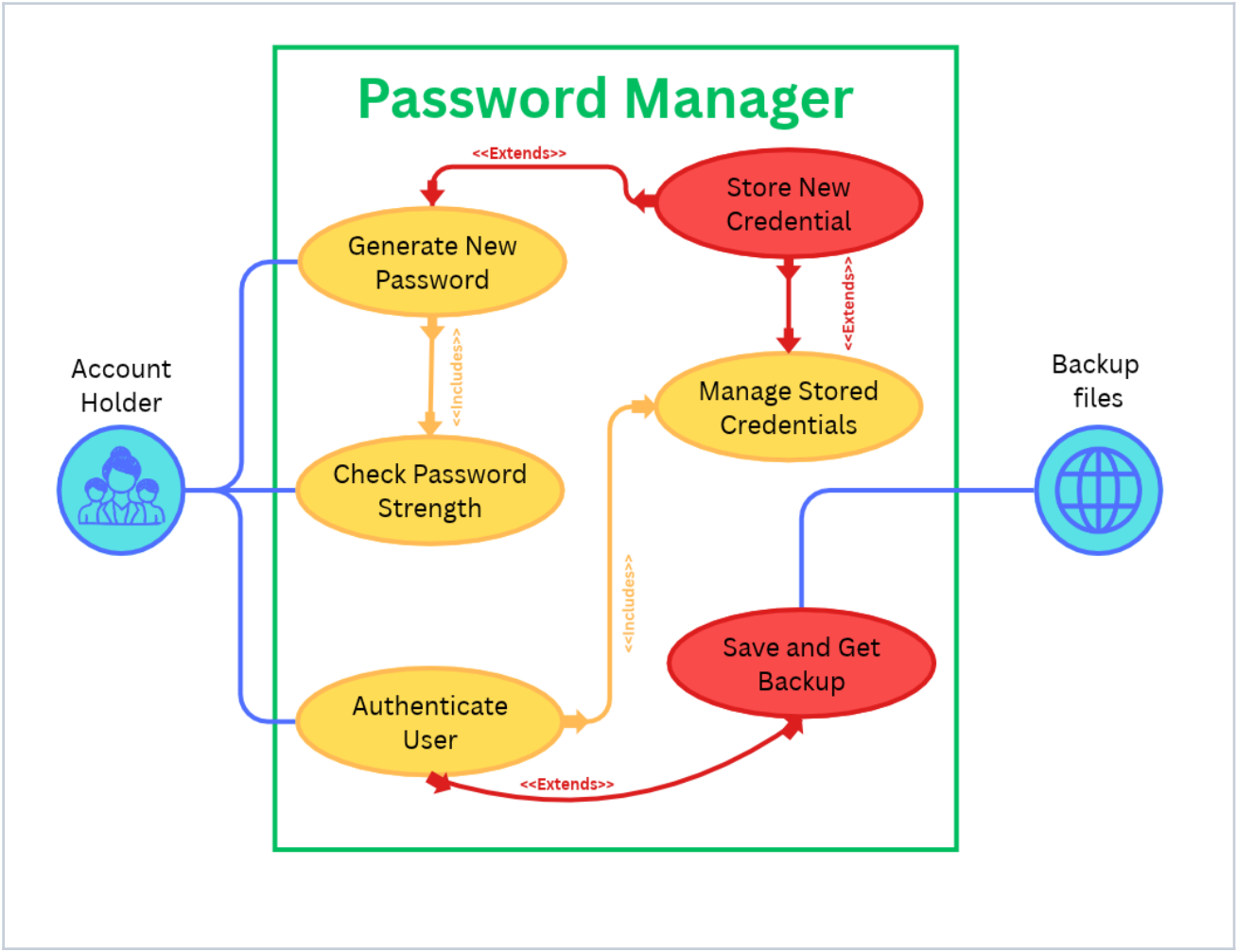
3.2 Quality requirements

ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	On the project’s test computers, 95% of vault-unlocking attempts shall be completed within 3 seconds, over 20 attempts.	High	Project Team	Measured across 20 attempts, where 19 are confirmed to finish within 3 seconds.
QR-02	After saving and closing the application, the saved fault shall contain 0 plaintext credentials, verified by examining the files with a text editor.	High	Project Team	Inspecting the saved file of encrypted vault files.
QR-03	The system shall automatically lock after 5 minutes of user inactivity, with the lock occurring within 5 seconds of the timeout, over 6 trial runs.	High	Project Team	Leave an unlocked vault inactive 6 times, confirming that the lock occurs between 5 minutes, and 5 minutes and 5 seconds after the last detected action.
QR-04	Out of 5 volunteer participants, 4 shall be able to add and retrieve a new credential within 2 minutes per task.	Medium	Project Team	5 volunteers are asked to complete 2 tasks using the system, and timed with each task.

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	System will be delivered by the end of the fall semester	Course Schedule	The deadline is set by our course and our final grade depends on it
CON-02	System will only use free services with no paid hosting	Team decision	We have no budget as college students so a paid service would be unaffordable for every member currently
CON-03	System will be demonstrable and our professor can access it without any special hardware	Project assignment	We are graded by demonstration so if we can't demonstrate our work we can't be assessed properly

3.4 Initial use cases



Expanded use cases

Field	Content
Identifier and name	UC-01: Generate New Password
Primary actor	Account Holder
Supporting actors	None
Preconditions — what must be true before it starts	The Account Holder is logged in
Trigger — what starts it	The Account Holder requests a new Password
Main success scenario — numbered steps	1. Account Holder requests a new password 2. Account Holder inputs restrictions such as length requirements, allowed characters, and necessary strength 3. Generate password by randomly selecting characters or words 4. Calculate password strength 5. Offer the new password to the Account Holder
Alternative and exception flows	• If the password strength is not sufficient per user requirements, repeat steps 3 and 4. Stop after 10 unsuccessful attempts and record not being able to generate such a password. • If the Account Holder rejects the offered password in step 5, either repeat steps 3 and 4 or quit out • Use generated password for a new credential

Field	Content
Postconditions — what is true when it succeeds	It generates a sufficiently strong password that can be used for creating a new credential
Requirements realised	FR-05

Field	Content
Identifier and name	UC-02: Store New Credential
Primary actor	Account Holder
Supporting actors	None
Preconditions — what must be true before it starts	The Account Holder is logged in
Trigger — what starts it	The Account Holder wants to store a new credential
Main success scenario — numbered steps	1. Account Holder marks what is needed, such as email, password, etc. 2. Account Holder fills out those requirements 3. Stores new credentials in the storage system
Alternative and exception flows	• The user uses the password generator for credential
Postconditions — what is true when it succeeds	The Account Holder has created a new credential which can be managed and used later
Requirements realised	FR-02

3.5 Requirements to use case traceability

Requirement ID	Use case(s) that realise it	If blank — why, and what you will do
FR-01	Authenticate User	
FR-02	Store New Credential	
FR-03	Manage Stored Credentials	
FR-04	Manage Stored Credentials	
FR-05	Generate New Password	
FR-06	Check Password Strength	
FR-07	Autofill Passwords	
FR-08	Autofill Passwords	
FR-09	Save and Get Backup	
FR-10	Save and Get Backup	

Removed 2 unnecessary requirements that restated FR-02 and FR-03 and added the use case “Store New Credential” for clarity and comprehension. Previously, it was only “Managing Stored Credentials” which was too complex and needed to be simplified with “Store New Credential”.

4 Use-Case Analysis

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC - 01	Generate New Password	This touches the security quality requirements directly, it cant be predictable and needs to pass the strength check.	Step 4:The random selection must be unpredictable. Step 3: The strength calculation must be consistent with what the user needs.
UC - 02	Store New Credential	One of the main focus of the project since almost all of our other use case depends on storing the new credentials.	Step 3: Saving to the storage system without losing any data if it fails.
UC - 03	Save and Restore Encrypted Backup	It realizes FR-09 and FR-10 and depends on the file system, so failures are visible to the account holder. The backup must contain 0 plaintext credentials (QR-02), and a restore must not damage the current vault.	Step 4, system build the backup from vault protected contents. Restore steps 4-8, system checks the file and opens with masterpassword and replaces the vault if it successed.

4.2 Expanded use cases

UC-01 Generate New Password

Primary actor	Account Holder
Supporting actors and external systems	None
TUCBW (this use case begins when)	The account holder requests a new password
TUCEW (this use case ends when)	The system confirms the password is accepted/cancelled

Preconditions	Account holder is logged in
Postconditions	A password that meets the holders restrictions is created and displayed to the user
Traces to requirements	FR-05 , QR- , CON

Actor input / action	System response
	(0) vault home with an option to generate password
(1) request new password	(2) ask for restrictions:length, characters, strength
(3) enters restrictions	(4) shows a generated password that fits
(5) accepts password	(6) confirms password is ready to use

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	3	Restrictions conflict (length is too short for the strength needed)	Systems tells holder what is conflicting and asks to change it	Rejoin at 3
A2	5	Holder rejects password	System generates another password and shows strength rating	Rejoin at 5
A3	5	Holder cancels	System tells holder no password was kept	ends

UC-02 Store New Credential

Primary actor	Account Holder
Supporting actors and external systems	None
TUCBW (this use case begins when)	The account holder chooses to add a new credential
TUCEW (this use case ends when)	The system confirms the credential is saved and shows in the list
Preconditions	Account holder is logged in
Postconditions	The new credential is stored and available to manage and autofill
Traces to requirements	FR-02 , QR- , CON

Actor input / action	System response
	(0) Shows list of stored credentials with an option to pass a new one
(1) Chooses to add a new credential	(2) shows what is needed: credential name, web address, user/email, and pass
(3) FIlls in details and submits them	(4) confirms credential is saved and shows on list

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	3	A required detail is missing	Systems names missing field and keeps details already entered	Rejoin at 3
A2	4	Saving fails	System tells holder nothing was saved and to reenter details	Rejoin at 3
A3	5	Holder cancels	System tells holder nothing was stored	ends

UC-03 save restore encrypted backup

Primary actor	Account Holder
Supporting actors and external systems	none
TUCBW (this use case begins when)	The account holder chooses to save a backup or to restore from a backup
TUCEW (this use case ends when)	The system confirms the backup was saved, or confirms the vault was restored and shows the credential list
Preconditions	The vault is unlocked
Postconditions	Save: a backup file exists at the chosen location and contains no readable credentials. Restore: the vault contains the credentials from the chosen backup.
Traces to requirements	FR-09, FR-10, QR-02

Actor input / action	System response
	(0) Shows the unlocked vault with options to save a backup and to restore from a backup
(1) Chooses to save a backup	(2) Asks where to put the backup file
(3) Provides the location	(4) Confirms the backup was saved and shows its name and time
(R1) Chooses to restore from a backup	(R2) Asks which backup file to use

(R3) Selects a backup file	(R4) Asks for the master password of that backup
(R5) Enters the master password	(R6) Warns that the vault contents will be replaced and asks for confirmation
(R7) Confirms	(R8) Confirms the vault was restored and shows the credential list

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	3	The location cannot be used	System names the problem and asks for another location	Rejoin at 3
A2	3	Holder cancels	System confirms no backup was saved	Ends
A3	4	The backup could not be written	System says nothing was saved and asks for a location	Rejoin at 3
A4	R3	The file is not a valid backup	System says so and asks for another file	Rejoin at R3
A5	R3	Holder cancels	System confirms the vault is unchanged	Ends
A6	R5	The master password does not open the backup	System says it is incorrect and asks again	Rejoin at R5
A7	R7	Holder declines the replacement	System confirms the vault is unchanged	Ends
A8	R8	The restore fails	System says the restore failed and the vault is unchanged	Ends

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
1	7 - Is there a post-condition a tester could turn into a test?	Although the post condition is present, it is not specific enough to make a test case	Made it more specific by rewording it to include how news of a finished password will be conveyed to a user
3	10 - Does any row name a technology, a store or a screen widget?	The expanded use case mentioned clicking on an input field, which is technology specific.	Replaced this with a generic step to avoid mentioning specific technological implementations
3	10 - Does any row name a	draft used “upload” and	Reworded to “save” and

	technology, store or widget?	“download”, which imply a web page	“restore”

5 Analysis Models

5.1 Domain model

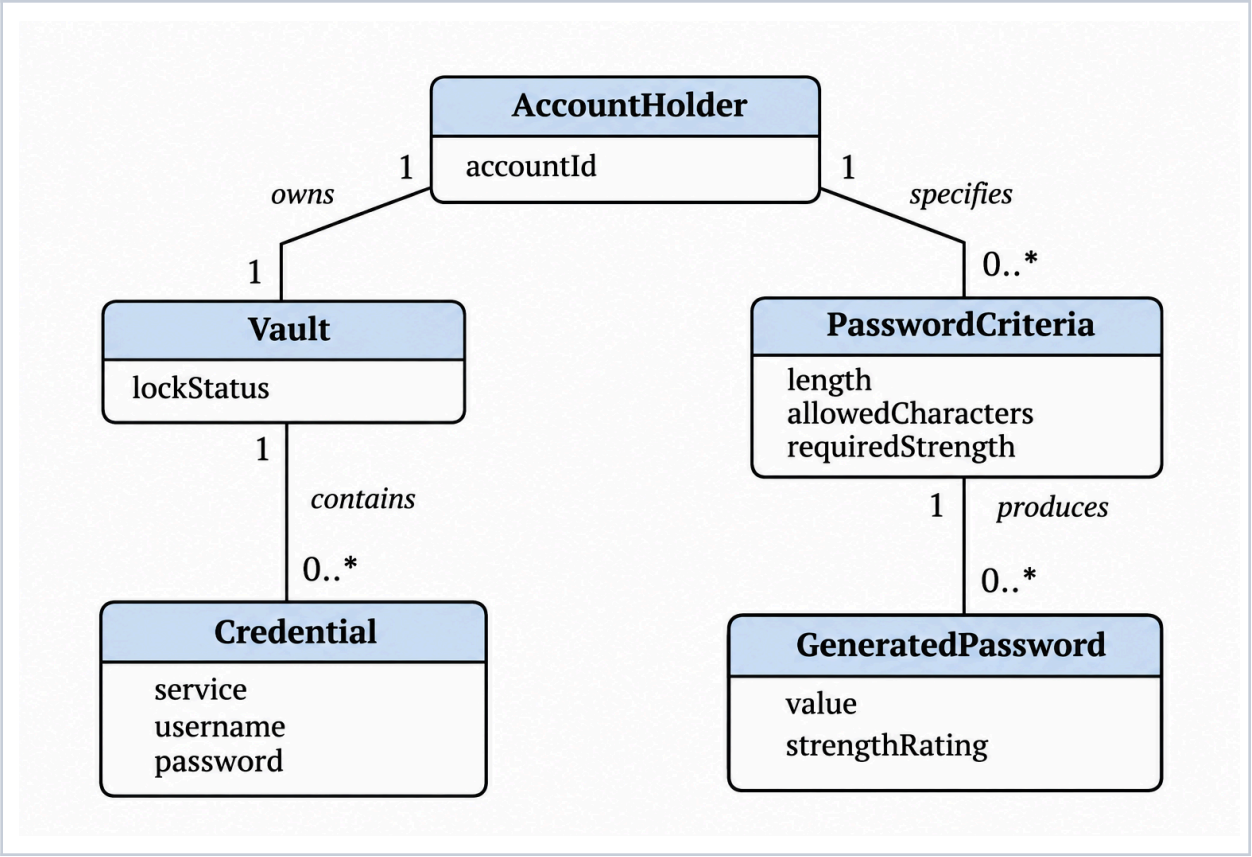


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
AccountHolder	A user who owns and accesses a password vault	accountId	Each AccountHolder owns one Vault and each AccountHolder specifies 0..* PasswordCriteria
Vault	The collection of credentials owned by an account holder	lockStatus	Each Vault belongs to one AccountHolder and each Vault contains 0..* Credentials
Credential	Login information stored for a	Service, username, and	Each Credential belongs

	specific service or website	password	to one Vault and each Vault contains 0..* Credentials
PasswordCriteria	Requirements selected when generating a password	Length, allowedCharacters, requiredStrength	An AccountHolder can define zero or more passwordCriteria. Each PasswordCriteria can produce 0..* GeneratedPassword
GeneratedPassword	A password created according to selected criteria	value,strengthRating	Each GeneratedPassword is produced from one PasswordCriteria

5.2 Object interaction models

5.2.1 UC-01 Generate New Password

(a) Non-trivial steps. Step (4) is non-trivial: The system stores the restrictions, verifies that they are not contradictory (A1), generates a password and evaluates its strength. Steps (1), (3) and (5) are actor actions. Only options and/or input is shown in steps (0) and (2), and step (6) only confirms the accepted password.

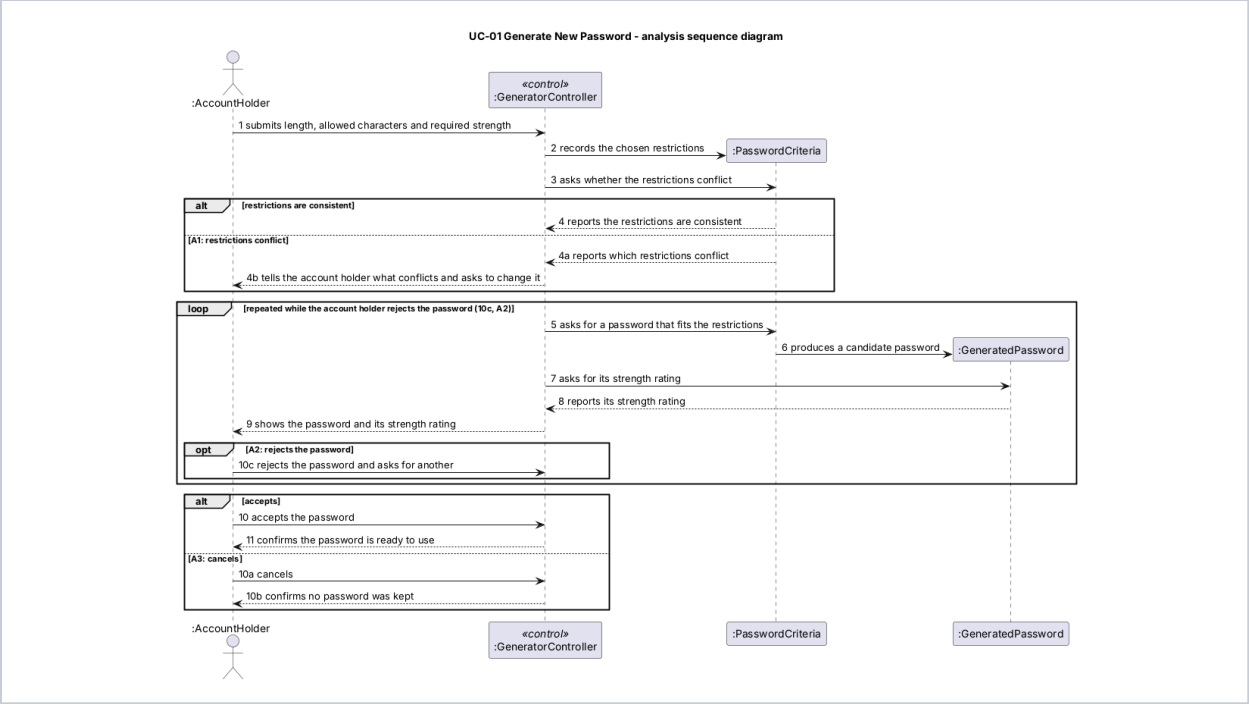
(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	AccountHolder	Submits the password restriction	Length, allowedCharacters, requiredStrength	GeneratorController
2	GeneratorController	Records the chosen restrictions	Length, allowedCharacters, requiredStrength	PasswordCriteria
3	GeneratorController	Ask whether the restrictions conflict		PasswordCriteria
4	PasswordCriteria	Report the restrictions are consistent		GeneratorController
4a	PasswordCriteria	Reports which restrictions conflict	Length, requiredStrength	GeneratorController
4b	GeneratorController	Tells the account holder what conflicts and ask to change it	Conflicting restrictions	AccountHolder
5	GeneratorController	Ask for a password that fits restrictions		PasswordCriteria
6	PasswordCriteria	Produces a candidate passwords	value	GeneratedPassword
7	GeneratorController	Ask for its strength rating	value	GeneratedPassword

8	GeneratedPassword	Reports its strength rating	StrengthRating	GeneratorController
9	GeneratorController	Shows the password and its strength rating	Value, StrengthRating	AccountHolder
10	AccountHolder	Accepts the password	value	GeneratorController
10a	AccountHolder	cancel		GeneratorController
10b	GeneratorController	Confirms no password was kept		AccountHolder
10c	AccountHolder	Rejects the password and asks for another	value	GeneratorController
11	GeneratorController	Confirms the password is ready to use	value	AccountHolder

GeneratorController is an interaction object that coordinates password generation and is in control of this interaction. When the account holder rejects the offered password (10c, A2), messages 5 to 9 repeat until the account holder accepts (10) or cancels (10a)

(c) Analysis sequence diagram.



5.2.2 UC-02 Store New Credential

(a) Non-trivial steps. Step (4) is not trivial: It verifies that the details are complete, verifies that the vault is unlocked, verifies for any duplication and creates the credential and stores it in the vault. Steps (2) and (0) only request input or only display the existing list, respectively, while steps (1) and (3) are actor actions.

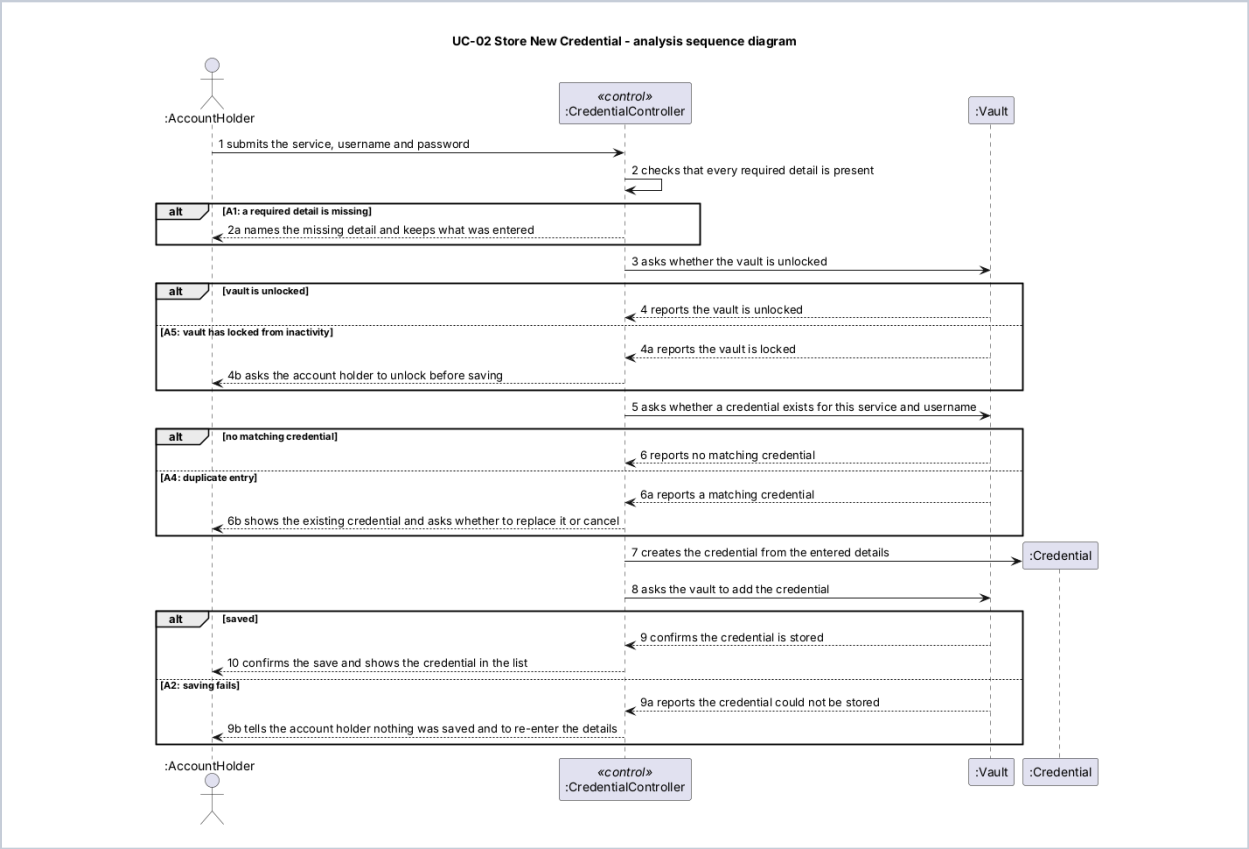
(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)

1	AccountHolder	Submits the new credential details	Service, username, password	CredentialController
2	CredentialController	Checks that every detail is present	Service, username, password	CredentialController
2a	CredentialController	Names the missing detail and keeps what was entered	Missing detail	AccountHolder
3	CredentialController	Ask whether the vault is unlocked	LockStatus	Vault
4	Vault	Reports the vault is unlocked	LockStatus	CredentialController
4a	Vault	Reports the vault is locked	LockStatus	CredentialController
4b	CredentialController	Ask the account holder to unlock before saving		AccountHolder
5	CredentialController	Ask whether a credential exist for this service and username	Service, username	Vault
6	Vault	Reports no matching credential		CredentialController
6a	Vault	Reports a matching credential	Existing credential	CredentialController
6b	CredentialController	Shows the existing credential and ask to replace or cancel	Existing credential	AccountHolder
7	CredentialController	Creates the credential from the entered details	Service, username, password	Credential
8	CredentialController	Ask the vault to add the credential	Credential	Vault
9	Vault	Confirms the credential is stored		CredentialController
9a	Vault	Reports the credential is not stored		CredentialController
9b	CredentialController	Tells the account holder nothing was saved and to reenter		AccountHolder
10	CredentialController	Confirms the save and show the credential in the list	Service, username	AccountHolder

CredentialController is an interaction object that coordinates storing a credential and is in control of this interaction.

(c) Analysis sequence diagram.



5.2.3 UC-03 Save and Restore Encrypted Backup.

(a) Non-trivial steps.Step (4) is non-trivial. The system checks whether a waiting period is active, checks the master password, opens the vault's contents, and updates the failed-attempt count. Steps (0) and (2) only display or request input, and steps (1) and (3) are actor actions.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	AccountHolder	Asks to save a backup		BackupController
2	BackupController	Asks where to put the backup file		AccountHolder
3	AccountHolder	Provides the location	location	BackupController
4	BackupController	Asks for the vault's protected contents		Vault
5	Vault	Provides its protected contents		BackupController
6	BackupController	Creates the backup file and stores it	location, contents	BackupFile
7	BackupFile	Reports it is stored		BackupController
7a	BackupFile	Reports it could not be		BackupController

		stored		
8	BackupController	Confirms the save, shows name and time	fileName, creationTime	AccountHolder
8a	BackupController	Says nothing was saved, asks for a location		AccountHolder
9	AccountHolder	Asks to restore from a backup		BackupController
10	BackupController	Asks which backup file to use		AccountHolder
11	AccountHolder	Selects a backup file	location	BackupController
12	BackupController	Asks for the backup's master password		AccountHolder
13	AccountHolder	Enters the master password	masterPassword	BackupController
14	BackupController	Asks the backup file to open with the password	masterPassword	BackupFile
15	BackupFile	Reports it opened, provides credentials	credentials	BackupController
15a	BackupFile	Reports it cannot be opened		BackupController
15b	BackupController	Says the file could not be opened, asks again		AccountHolder
16	BackupController	Warns contents will be replaced, asks to confirm		AccountHolder
17	AccountHolder	Confirms		BackupController
17a	AccountHolder	Declines		BackupController
17b	BackupController	Confirms the vault is unchanged		AccountHolder
18	BackupController	Asks the vault to replace its contents	credentials	Vault
19	Vault	Reports it was replaced		BackupController
19a	Vault	Reports it could not be replaced		BackupController
20	BackupController	Confirms the restore, shows credential list	Service, username	AccountHolder
20a	BackupController	Says the restore failed, vault unchanged		AccountHolder

(c) Analysis sequence diagram.

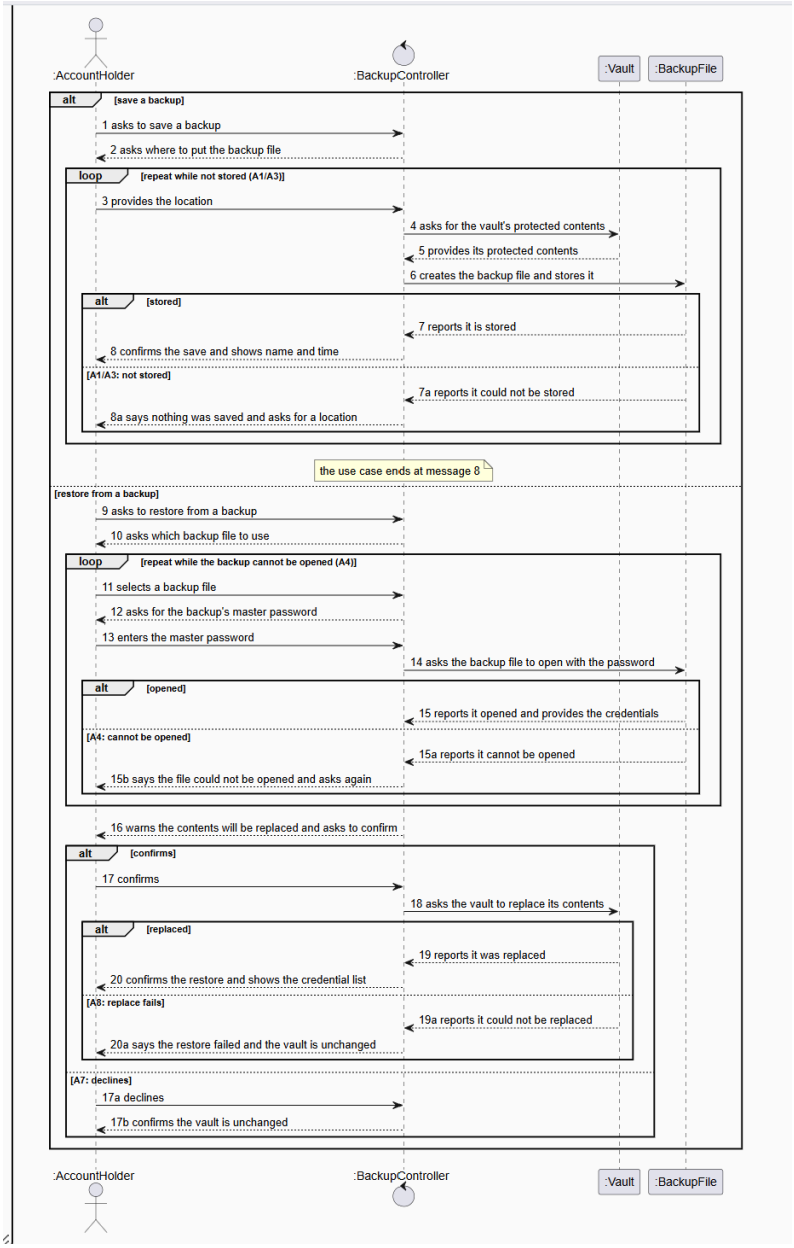


Figure 5.2.2 Analysis sequence diagram for UC-02; message numbers match the scenario table

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Fail, the supporting actor in UC-03, the extension framework, is not included in the domain model	

K2 Every object on a sequence diagram is a domain class or a declared interaction object	Pass after fix. The controllers are not domain classes; they are declared as interaction objects in 5.2.1 and 5.2.2.	
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass. Although note that 10 or 10a should be in the loop or added a specific check for whether the AccountHolder accepts the rejects the password in 5.2.1. Also note that in 5.2.2 doesn't allow for multiple credentials entries like email, backup pin, etc. 5.2.2 also has the expected pathway in alternate boxes.	
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Pass, every step has a corresponding row	
K5 Every analyzed use case traces to at least one functional requirement	Fail, UC-03 doesn't match to FR-05, there's lacking requirements for quality and constraints for all use cases in 4.2	
K6 The last messages of each diagram make the use case postcondition true	Pass, a credential is made in 5.2.2 and a password is generated in 5.2.1.	

6 High-Level Design (Architecture)

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	An account holder submits a valid master password to unlock the vault on a project test computer.	Authentication and vault decryption	Vault is decrypted and credential list is shown	At least 19/20 login attempts finish within 3 seconds.	QR-01
QAS-02	Security	Someone opens the vault in a text editor after the application is closed	Vault storage and encryption	The contents inside are unreadable and need to be decyphered	0 plaintext credentials found in saved files after opening	QR-02
QAS-03	Security	An unlocked vault receives no	Session lock	The vault locks and the	Lock happens between 5min -	QR-03

		account holder input for 5 minutes		decrypted data is not showing and the master password is needed again	5min 5sec after the last action, should test and work 6/6 times	
QAS-04	Useability	A new user is asked to add a credential and be able to receive it	UI and controller components	New user is able to complete the task without needing help	⅘ people are able to complete the task in a span of 2 minutes	QR -04

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered	QAS-01 - 04 CON-01 - 03	Strictly layered and the vault is tied to one device which works because multiple devices use is out of scope	Chosen
Cloud hosted	Multi device access and easy backup	Needs hosting which breaks con-02	Rejected
Microservices	Independent deployment and scaling	Too much for one user on one device. Might threaten con-01	Rejected

6.3 Component view

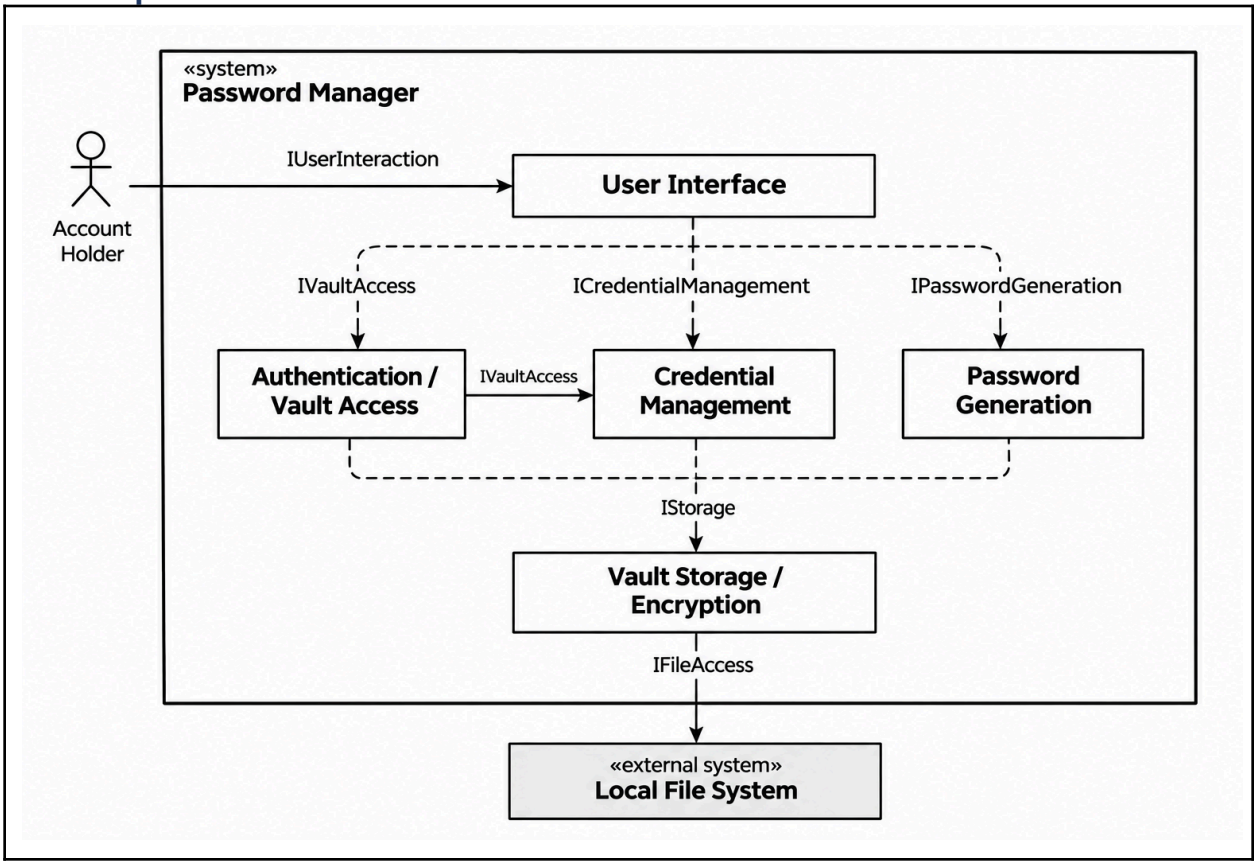


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
User Interface	Receives account-holder requests and presents results from the password manager	UserInteraction	Authentication, CredentialManagement, PasswordGeneration	UC-01,UC-02, QAS-04
Authentication/ Vault Access	Verifies access to the vault and manages locked/unlocked state	VaultAccess	VaultStorage	FR-01, QAS-01, QAS-03
Credential Management	Creates, retrieves, edits and deletes stored credentials	CredentialManagement	VaultAccesss, VaultStorage	UC-02, FR-02, FR-03, FR-04, FR-07
Password Generation	Generates passwords according to user-selected restrictions and evaluates their strength	PasswordGeneration	UserInteraction	UC-01, FR-05, FR-06
Vault Storage/ Encryption	Stores credentials in encrypted local storage and retrieves them for authorized components	SecureVaultSotrage	Local File System	QAS-02, FR-02, FR-03

6.4 Deployment view

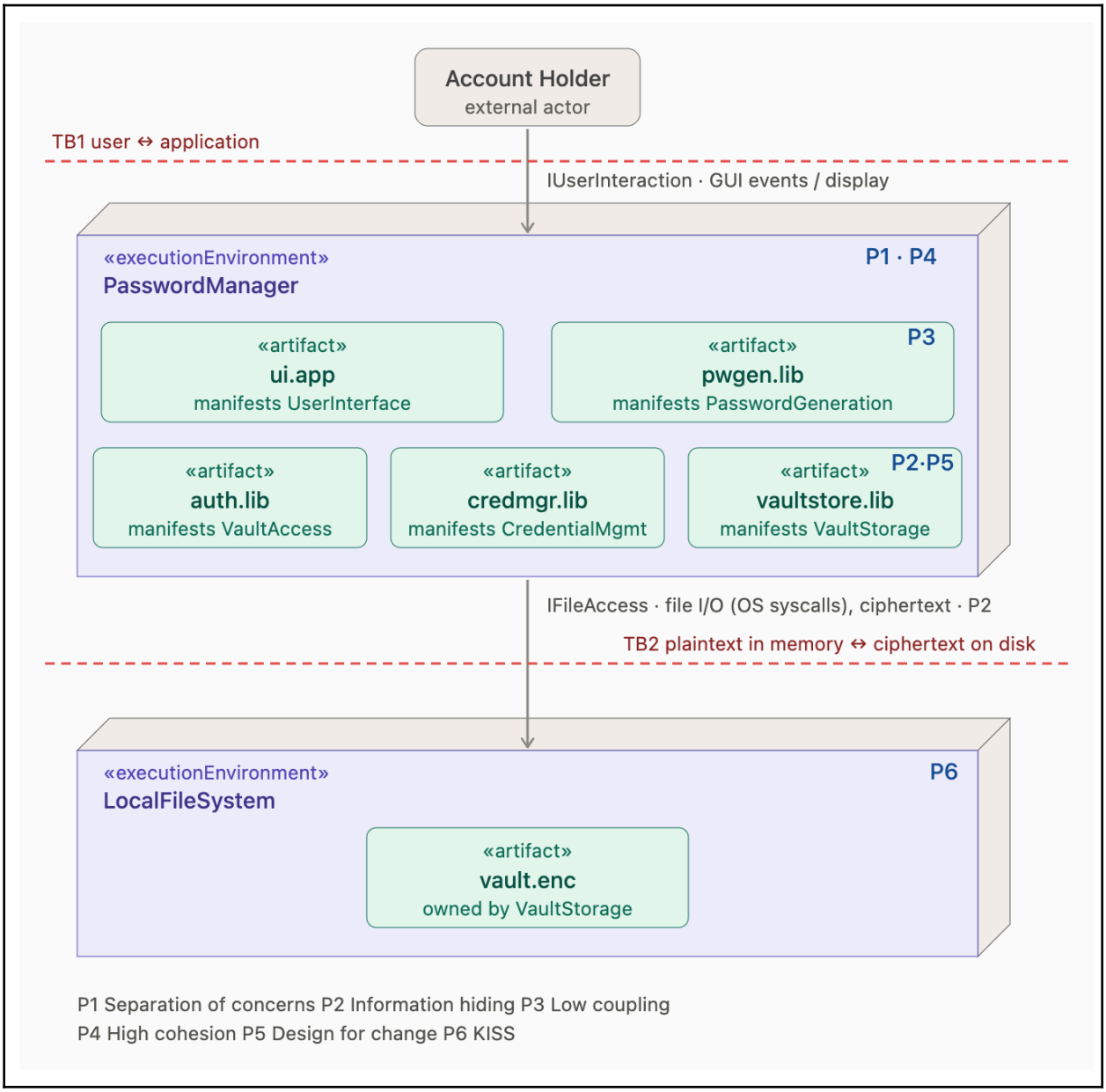


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
PasswordManager	Execution Environment with UI.app and Pwgen.lib, auth.lib, credmgr.lib and vaultsore.lib	LocalFileSystem	TB1
LocalStorage	Device - with vault.enc environment	N/A	TB2

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
AccountHolder	User Interface	External - not part of the system
Vault	Vault Storage/Encryption	Split - Credentials in VaultStorage/Encryption. Lock status is in authentication/vault Access
Credential	Credential Management	Not split - in it's own component
PasswordCriteria	Password Generation	Merged with GeneratedPassword
GeneratedPassword	Password Generation	Merged with PasswordCriteria
GeneratorController	User Interface	Split: generation, conflict check go to Password Generation

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	P1 - Only one artifact per responsibility in Password Manager	All components of 6.3 are represented via one artifact
Information hiding	P2 - Only vaultstore.lib has vaul.enc.	IStorge is the only way to reach it. IFileAccess is the only way into the file system.
Low coupling	P3 - wgen.lib has no dependency on the vault artifacts	PasswordGeneration only needs UserInteraction
High cohesion	P4 - Each artifact only has one component with a single responsibility	Each component handles something independant
Design for change	P5 - Encryption format is from vaultstore.lib	Easy to change because is modular so change mainly affects vaultstore.lib
Keep it simple (KISS)	Just two nodes	Deployment view

7 Design Rationale

7.1 Architecture decision records

ADR-01 Use Layered Architecture

Status	Accepted
Date and owner	11/7/2026 - Project Team
Context (drivers)	This system requires a maintainable, secure architecture that reliably allows for separate responsibilities and simplified development. Using a layered architecture will support the project's quality attribute scenarios (QAS-01 to QAS-04) and constraints (CON-01 to CON-03). The application is intended for a single user on one device, making multi-device synchronization and/or service-oriented architecture unnecessary.
Options considered	Layered Architecture - Separate presentation, application logic, and data accessing into multiple layers. Cloud-Hosted Architecture - Provides centralized storage, and allows for multi-device capabilities, but would require hosting services. Microservices Architecture - Separates functions into deployable services, introducing complications and several new requirements.
Decision	Maintain a layered architecture with separate user interface, application logic, and data management. Each layer communicates through developed interfaces.
Consequences	Favors - Maintainability, modularity, separate concerns. Move - Organize application functions into distinct layers with different responsibilities. Cost - Additional structure decisions and communication between design layers.
Evidence	Implementation and prototype evidence is not available. This decision is based on the current project scope, architectural constraints, and comparisons made in section 6.2.
Related	QAS-01, QAS02, QAS-03, and QAS-04. CON-01, CON-02, and CON-03.

ADR-02 Use Local-Only Storage

Status	Accepted
Date and owner	11/7/2026 - Project Team
Context (drivers)	The password management system is intended for a single user operating a single device. Storing vault information locally avoids dependency on external services, and reduces the operational complexity of using cloud-based storage. This decision aligns with the project's constraints on hosting.
Options considered	Local Storage - Store the password vault on the user's device using a local database/file. Cloud-Based Storage - Store vault information on a remote server, allowing centralized access and potential synchronization across multiple different devices for users. Mixed Storage - Maintain an encrypted local vault that also periodically updates/uploads the information with a remote service.
Decision	Use local-only storage for the password vault. Sensitive vault data will remain only on the user's device without requiring a remote server or cloud database. A simple, application-embedded database system created with SQLite may be used.

Consequences	Favors - Security, simplicity, independence from external services. Move - Implement localized, persistent storage and ensure that vault data is protected from unauthorized access. Cost - Users will not have automatic multi-device synchronization, and device failure or data corruption results in permanent loss of vault information, unless a backup mechanism is added.
Evidence	Implementation and Prototype evidence is not available yet. The decision is supported by the project's scope regarding single-device systems, and section 6.2. Open question: What local storage format best supports encrypted vault data, and allows for reliable persistence and recovery?
Related	ADR-01. CON-02. CH-01, CH-02, CH-03.

ADR-03 Encrypt Vault Data Using Key From Password

Status	Accepted
Date and owner	11/7/2026 - Project Team
Context (drivers)	The application stores information like the user's account credentials and passwords. This information must remain protected, even if unauthorized access occurs with the device's stored vault. Since the application is designed for local use, a remote service cannot perform encryption and decryption.
Options considered	Password-Based Encryption - Derive an encryption key from the user's passkey and use it to encrypt and decrypt the vault content. Unencrypted Local Storage - Store credentials directly in the local database or files.
Decision	Encrypt sensitive vault contents using a key derived from the user's main password. A password-based key function should be used to obtain an encryption key, and use an algorithm to protect the stored vault data against unauthorized reading and writing.
Consequences	Favors - Confidentiality, data integrity, security. Move - Implement password-based key derivation and encryption for vault data. Cost - Encryption and decryption introduces computational requirements, and implementation complexity. Losing the main password for the vault may also lead to the whole vault becoming unrecoverable if no other recovery mechanisms exist.
Evidence	Implementation and cryptographic testing has not been performed. This decision is based on the project's established principles involving security. Open question: What authenticated encryption algorithm, and key derivation strategy should be chosen, and how will vault recovery be handled?
Related	ADR-01, ADR-02. QAS-01, QAS-02. CH-01.

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Maintainability (QAS-04, CON-01)	Layered architecture with separate UI, app logic and data management	Extra structure and interlayer communication to design and maintain	ADR -01
Security and Independence of external services (QAS-02, CON-02)	Local only vault storage on users device, with no server on cloud	No multi device sync. If device fails or gets corrupted permanently lose the vault unless backup is made	ADR -02
Confidentiality of stored credentials (QAS-02)	Encrypt the vault with a key derived from the master password	Key derivation and decryption time counts against the 3 sec unlock target (qas-01) and a forgotten master pass makes the vault unrecoverable (A1)	ADR -03

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01 Backup Test
Risk or decision it addresses	R-5 / ADR-01 / QAS-01 and QAS-04
Question (answerable yes or no)	Does the user run into any unpredicted behavior?
Set-up	Completely empty vault with a premade account
How it was measured	1. Login 2. Create two unique credentials 3. Create a backup 4. Delete one credential and create another unique credential 5. Load backup 6. See if the backup successfully loaded the deleted credential and deletes the new credential
Result (numbers)	Number of errors or unexpected behavior encountered
What it changed or confirmed	Confirms R-5 with no major errors, confirms ADR-01 with successful layer communication, confirms QAS-01 with login, and confirms QAS-04 with multiple credential creations.
Where to see it	Log file of actions and errors

Spike ID and title	S-02 Encryption Test

Risk or decision it addresses	R-2 / ADR-02 / QAS-02
Question (answerable yes or no)	Can somebody see unencrypted credentials without being logged in?
Set-up	A logged out account with a vault that contains at least one credential
How it was measured	1. Navigate to where credential data is stored 2. Open credential files 3. Check whether files are legible/unencrypted
Result (numbers)	Number of unencrypted files
What it changed or confirmed	Confirms R-2 and QAS-02 with unintelligible credential files and confirms ADR-02 as the files are accessible from the computer.
Where to see it	Picture of credential files

Spike ID and title	S-03 Account Creation
Risk or decision it addresses	R-5 / ADR-03 / QAS-01, QAS-03, and QAS-04
Question (answerable yes or no)	Does the user run into any unpredicted behavior?
Set-up	No account beforehand
How it was measured	1. Create account 2. Create one credential 3. Wait for 5 minutes and get logged out 4. Log in 5. Check if credential is still there after logging back in
Result (numbers)	Number of errors or unexpected behavior encountered
What it changed or confirmed	Confirms R-5 with no major errors, confirms ADR-03 by creating a main password, confirms QAS-01 logging in, confirms QAS-03 by waiting to be timed out, and confirms QAS-04 by creating a credential.
Where to see it	Log file of actions and errors

Spike ID and title	S-04 Different Encryptions
Risk or decision it addresses	R-6 / ADR-03
Question (answerable yes or no)	Is encryption different between accounts?
Set-up	Have 2 different accounts with the same master password
How it was measured	1. Create an identical credential in both accounts

	2. Sign out from both accounts 3. Compare encrypted files for the specific credential data
Result (numbers)	0 - files are identical 1 - files are different
What it changed or confirmed	Confirms R-6 if the encryption is different between the credentials confirms ADR–03 that with the same master password, the encryptions are still different
Where to see it	Picture of credential files

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	Authenticate User / Unlock Vault	AccountHolder, Vault	User Interface: Authentication or Vault access and Vault Storage or Encryption	QAS-01, QAS-03, ADR -01	Section 6.1 (QAS-01, QAS-03); Section 6.3; ADR-03
FR-02	UC-02 Store New Credentials	AccountHolder, Credential Vault, CredentialController	User Interface; Credential Management; Vault Storage / Encryption	QAS-04, ADR-01, ADR-02, ADR-03	Section 4.2 UC-02; Section 5.2.2; Section 6.3
FR-03	Manage Stored Credentials - Retrieve	AccountHolder, Credential, Vault	Credential Management; Vault Storage / Encryption	QAS-04, ADR-02, ADR-03	Section 3.5; Section 5.1; Section 6.3
FR-04	Manage Stored Credentials - Edit	AccountHolder, Credential, Vault	Credential Management	QAS-04, ADR-02	Section 3.5; Section 5.1; Section 6.3
FR-05	UC-01 Generate New Password	AccountHolder, PasswordCriteria, GeneratePassword	User Interface; Password Generation	ADR-01	Section 4.2 UC-01; Section 5.2.1; Section 6.3
FR-06	Check Password Strength	PasswordCriteira, GeneratePassword	Password Generation	ADR-01	Section 5.1; Section 6.3

FR-07	Manage Store Credentials -Delete	AccountHolder, Credential, Vault	Credential Management	ADR-01, ADR-02	Section 3.1; Section 6.3
FR-08 - Withdrawn (CH-03)	Autofill Passwords - withdrawn	—	—	CH-03	Section 0.2
FR-09	Save Encrypted Backup	Vault	Vault Storage / Encryption	QAS-02, ADR-02, ADR-03	Section 3.1 and 6.3; ADR-02
FR-10	Restore Encrypted Backup	Vault	Vault Storage / Encryption	QAS-2, ADR-02, ADR-03	Section 3.1 and 6.3; ADR-02
QR-01	Authenticate User / Unlock Vault	AccountHolder, Vault	Authentication / Vault Access; Vault Storage / Encryption	QAS-01, ADR-01, ADR-03	QAS-01 in Section 6.1; Section 6.3; ADR-03
QR-02	N/A – quality requirement	Vault, Credential	Vault Storage / Encryption	QAS-02, ADR-02, ADR-03	QAS-02 in Section 6.1; Section 6.3; ADR-02/03
QR-03	Authenticate User / Unlock Vault	AccountHolder, Vault	Authentication / Vault Access	QAS-03, ADR-01	QAS-03 in Section 6.1; Section 6.3
QR-04	UC-02 Store New Credential; Retrieve Stored Credential	AccountHolder, Credential, Vault	User Interface; Credential Management	QAS-04, ADR-01	UC-02 in Section 4.2; QAS-04 in Section 6.1; Section 6.3
CON-01	N/A – project-wide constraint	N/A	All implementation components	ADR-01	Section 6.2; ADR-01
CON-02	N/A – project-wide constraint	N/A	Architecture-wide; Vault Storage / Encryption	ADR-01, ADR-02	Section 6.2; ADR-02
CON-03	N/A – project-wide constraint	N/A	All implementation components	ADR-01	Section 6.2; Section 6.4; ADR-01

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	FR-08 autofill we withdrew it in ch-03 so it has no component by design	We keep it in the matrix marked withdrawn, that way our traceability isn't lost. The web browser and autofill use case were also removed with it so no component is needed
Components with no requirement, QAS or ADR	none , we have at least one requirement in every component in 6.3	n/a
Use cases not analyzed (and why that is acceptable)	Authenticate user/ unlock vault, manage stored credentials, check password strength	Check password: acceptable because we analyse it inside UC-01 Manage stored credentials: acceptable because retrieve, edit and delete use the same credential. Where vault and credential/controller objects we had analyzed them in UC-02 Authenticate user: not acceptable to leave analyzed

The withdrawn Autofill use case (CH-03) was replaced as our third analyzed use case by UC-03 Save and Restore Encrypted Backup (FR-09, FR-10, QR-02), which is expanded in sections 4.1, 4.2 and 5.2.3. This gives us three analyzed use cases: UC-01, UC-02 and UC-03. We also replaced the old section 3.5 matrix with a pointer to section 9.1, because it mapped FR-07 to Autofill.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Shubhi Kohli	Requirements and Use-case Analysis	Refer to the documented requirements and use cases in P1 to compare design options and make the best decisions.
Jacob Weikum	System Architecture Design	Explain the technical compromises and overall system changes to help the team keep a maintainable design.

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Michael Taylor	Design Rationale, Security Analysis	Evaluate the design risks, and ensure decisions prioritize the project requirements involving security.
Samantha Escalada	System Modeling, Design Documentation	Use system diagrams and models to sort out misunderstandings and compare design choices.
Dongho Lee	Data and Traceability Analysis	Check proposed merges and project decisions to design elements and tests.
Derek Chai	Prototype and Technical Lead	Use prototype results and technical feasibility to analyze competing approaches towards project goals, and ensure the team reaches evidence-based conclusions.

10.2 Plan to P3

Milestone	Date	Owner	Done when
P3 document analysis	11/13/2026	Group	Team meeting discussing P3 structure and template. Done when delegations are given out.
Account System is completed	11/18/2026	Group	A user can create an account, log in, and log out. There are different accounts that don't overlap.
Password generator is completed	11/20/2026	Group	A password is generated based on constraints.
Credentials are able to be made and deleted	11/25/2026	Group	Credentials are able to be made and deleted and can use the password generator.
Credentials are encrypted	11/30/2026	Group	Credential data is encrypted
Export credentials	12/2/2026	Group	Able to export and import encrypted credentials.
Parts work together	12/3/2026	Group	An account holder is able to log in, create credentials, use the password generator, export credentials, and credential files should be

			encrypted.
Final Review and P3 is submitted	12/9/2026	Group	Final reviews and touches are done and P3 is submitted

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R-1: One team member is unavailable or ceases to participate before P2	Everybody has contributed something to both P1 and P2	Likelihood - Low Impact - High	Everyone contributes towards the github via branches and reviews, and sections are divided out.
R-2: Credential encryption is not completed or performed in a secure manner.	More research and information has been found	Likelihood - Medium Impact - High	The encryption and decryption solely goes through Java's standard library for security which allows for secure encryption.
R-3: Work on optional integrations in the context diagram delays the MUST requirements.	We've redefined our musts and shoulds and made our timeline more feasible	Likelihood - Low Impact - Medium	We will work on all Must cases before any other use cases.
R-4: The team runs short on time before the end of semester.	We have had our first meeting that let us plan P2	Likelihood - Low Impact - High	Group meetings and free communication allow for check ups and reprioritization. This allows us to point out flaws and finish before the deadline.
R-5: A change merged to main breaks working features close to a deadline.	Everybody has gotten more experience with merging and checking branches	Likelihood - Medium Impact - Low	Merges can be reverted plus merge requests must be reviewed.
R-6: Password encryption doesn't change between accounts	New so N/A	Likelihood - Low Impact - High	Use Java's standard library and other factors like time to create differentiation.

10.4 Repository

https://github.com/MVT04/3354-Team-09-Project

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Derek Chai	Claude	To generate diagrams for 5.2	Checked each row against the team's 4.2 steps and the 5.1 domain model
Samantha Escalada	Claude	Diagrams for 5.2	Updated with the new use case since removing autofill, same 4.2-5.1
Dongho Lee	ChatGPT	Diagrams for 5.1; 6.3	Used to help create diagrams and review/refine Sections 5.1, 6.3, 9.1

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Which browser will support autofill?	P1 baseline	Derek Chai - Oct 25	(Closed) Autofill was removed from the implementation scope under CH-03	N/A
Q2	Will backup and restore be included in the first release?	P1 baseline	Shubhi Kohli - Oct 25	(Closed) Yes, Backup and restore are included as FR - 09 and FR - 10 and the recovered by UC - 03	N/A
Q3	Should the backup use case cover both saving and restoring backup?	P1 baseline	Dongho Lee - Oct 25	(Closed) Yes, UC - 03 covers both saving and restoring encrypted backups.	N/A
Q4	Should we use SQLite or an encrypted local file for vault storage?	P2 / ADR -02	Michael Taylor - Nov 20	Open	We will use local -only storage until the exact format is selected
Q5	What should happen if the	P2 / ADR	Michael Taylor	Open	The vault will use

	user forgets the master password?	-03	- Nov 20		the master password for encryption until a recovery method is decided.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
Samantha Escalada	Samantha Escalada	11/08/26
Dongho Lee	Dongho Lee	11/08/26
Derek Chai	Derek Chai	11/08/26
Shubhi Kohli	Shubhi Kohli	11/08/26
Jacob Weikum	Jacob Weikum	11/8/2026
Michael Taylor	/Michael Taylor/	11/8/2026